TRI-CA

Experimental Platform

Installation & Configuration Tutorial

A complete guide for researchers configuring human–AI interaction experiments

1. System Overview

TRI-CA is an open-source experimental platform designed to support controlled studies involving Large Language Model (LLM)-based conversational agents. The platform enables researchers to design, deploy, and manage experiments in which human participants interact with conversational agents under different treatment conditions.

The system provides infrastructure for:

- Structured human–AI interaction experiments
- Random assignment to treatment conditions
- Administration of pre- and post-interaction surveys
- Logging of conversation transcripts and participant actions
- Storage of experimental data in a centralized database

TRI-CA is implemented as a Node.js web server application and integrates several components:

Component	Description
Front-end Interface	Web interface where participants interact with the conversational agent
Application Server	Node.js server managing experiment flow and logic
LLM Integration	API integration with OpenAI or other LLM providers
Database Layer	Stores interaction logs, survey responses, and experiment metadata
Configuration Layer	CSV-based configuration files used to define experimental design
Researcher Interface	Backend tools for configuring experiments and extracting data

The system architecture enables researchers to configure experiments without modifying the core application code, supporting transparency and reproducibility.

2. Repository Structure

The TRI-CA platform is publicly available via GitHub. The repository contains the full implementation of the platform as well as documentation and configuration templates.

```
tri-ca/
│
├── server/                Node.js server application
├── frontend/              Participant interface
├── experiment_config/     CSV experiment configuration files
├── prompt_bank/           Hidden prompts for conversational agents
├── task_bank/             HTML task descriptions
├── prerequisite/          Participant access codes
├── analysis/              Python data extraction scripts
```

```
├── docs/                   Documentation and tutorial
└── README.md               Installation instructions
```

3. System Requirements

3.1 Software Requirements

Component	Recommended Version
Node.js	v18 or later
npm	v9 or later
Python	v3.9 or later
Git	Latest version

3.2 External Services

The system requires access to a conversational AI service. Researchers must obtain their own API credentials from one of:

- OpenAI API
- Azure OpenAI
- Another compatible LLM provider

3.3 Database

TRI-CA requires a relational database for storing experimental data. The recommended database is PostgreSQL.

4. Installation

The estimated installation time is 10–20 minutes.

Step 1: Clone the Repository

Open a terminal and run:

```
git clone [repository link]
cd tri-ca
```

Step 2: Install Dependencies

```
npm install
```

This command installs all required server-side dependencies.

5. Configuring LLM Access

Step 1: Obtain an API Key

Create an account with your chosen LLM provider (e.g., OpenAI) and generate an API key through the provider dashboard.

Step 2: Configure Environment Variables

Create a .env file in the root directory of the project with the following contents:

```
OPENAI_API_KEY=your_api_key_here
DATABASE_URL=your_database_connection_string
```

⚠ **Note:** API keys should never be committed to public repositories. Add .env to your .gitignore file.

6. Server Deployment

The TRI-CA system must be deployed on a web server so that participants can access the experiment. The platform can run on any Node.js-compatible hosting environment including Heroku, AWS, DigitalOcean, or institutional research servers.

Example: Deploying to Heroku

```
heroku create tri-ca-experiment
git push heroku main
```

Once deployed, the application will be accessible through a public URL which participants will use to access the experiment.

7. Database Configuration

TRI-CA stores all experimental data in a relational database (PostgreSQL recommended).

Example: Heroku PostgreSQL Setup

```
heroku addons:create heroku-postgresql:hobby-dev
heroku config
```

The system uses the DATABASE_URL environment variable to connect to the database. Before launching, verify that the system can:

- Store participant responses
- Record conversational interactions
- Retrieve experiment configuration files

8. Experiment Configuration

Experimental design is defined through CSV configuration files in the experiment_config/ directory. These files control survey structure, participant instructions, treatment conditions, agent prompts, and task descriptions — without requiring any changes to application code.

8.1 Question Bank (CSV)

Each row in the Question Bank CSV corresponds to one survey item.

Field	Description
block_name	Name of the survey block containing the question
allow_block_permutation	Allows randomization of block order
allow_question_permutation	Allows randomization of questions within a block
question_name	Internal identifier
question_text	Text shown to participants
is_text	Indicates open-ended response
is_likert	Indicates Likert-scale question
is_grouped_likert	Indicates matrix-style Likert question
is_multi_choice	Indicates multiple-choice question
is_label	Instructional label with no response
options	Response options or scale values

8.2 Experiment Description File (CSV)

Controls the text displayed throughout the experiment interface.

Field	Description
page	Page identifier
title	Page title
header	Section heading
body1	Main instructions or explanatory text

8.3 Treatment Group Configuration (CSV)

Each row represents one experimental condition. This file drives random assignment of participants.

Field	Description
treatment_group	Identifier for experimental condition
user_name	Label displayed for participant
user_avatar	Participant avatar
agent_name	Conversational agent name
agent_avatar	Conversational agent avatar
hidden_prompt	System prompt controlling agent behavior
user_task_description	Task instructions for participants
user_pre_questions	Pre-interaction survey block
user_post_questions	Post-interaction survey block

9. Supporting Components

9.1 Hidden Prompt Repository

The prompt_bank/ folder contains .txt files defining system prompts used to guide conversational agent behavior. These prompts influence the agent's responses but are not visible to participants, ensuring experimental control.

9.2 Task Description Repository

The task_bank/ folder contains HTML files describing the tasks participants complete during the experiment. Different treatment groups may reference different task descriptions.

9.3 Participant Access Control

The prerequisite/ folder contains a CSV file with participant access codes. Access codes allow researchers to:

- Control access to the experiment
- Prevent duplicate participation
- Maintain anonymous participant identifiers

10. System Testing

Before launching the experiment, conduct comprehensive testing to verify all components are working correctly. Testing should be conducted both locally and on the deployed server environment.

Testing should verify:

1. Server deployment is accessible at the expected URL
2. Database connectivity — participant data is being stored correctly
3. Treatment group assignment — participants are randomly assigned
4. Survey rendering — all question types display correctly
5. Conversational agent responses — the LLM API is responding
6. Interaction logging — conversation transcripts are being recorded

11. Running the Experiment

Once testing is complete, the experiment can be launched. Researchers should:

7. Distribute access codes to participants
8. Provide the experiment URL
9. Instruct participants to enter their assigned access code

Participants will then complete the following sequence:

10. Pre-experiment survey
11. Interaction with the conversational agent
12. Post-experiment survey

All participant interactions and responses are automatically recorded.

12. Data Collected

Data Type	Description
Conversation logs	Full transcripts of participant–agent interaction
Clickstream data	User navigation and interaction events
Survey responses	Pre- and post-experiment questionnaires
Treatment assignment	Participant experimental condition
Timestamps	Event timing for interaction analysis

All data are stored anonymously and linked only to participant access codes.

13. Data Extraction

After data collection is complete, researchers can export the dataset using the Python script located in the analysis/ folder:

```
python extract_data.py
```

The script compiles all experimental data into structured datasets suitable for statistical analysis.

14. Frequently Asked Questions

Q: Can I use a different LLM provider besides OpenAI?

Yes. TRI-CA supports any LLM provider with a compatible REST API. Update your .env file with the appropriate API key and configure the integration module accordingly.

Q: Can I modify the experiment without changing application code?

Yes. This is a core design principle of TRI-CA. All experimental parameters — including survey questions, treatment groups, agent prompts, and task descriptions — are controlled via CSV configuration files.

Q: How many treatment groups can I define?

There is no hard limit on the number of treatment groups. Each row in the Treatment Group CSV defines one condition. The system will randomly assign participants across all defined groups.

Q: How do I prevent participants from completing the experiment more than once?

Each participant is assigned a unique access code in the prerequisite/ folder. The system tracks usage of these codes and prevents duplicate submissions.

Q: What format is the exported data in?

The Python extraction script compiles data into structured tabular datasets (CSV) suitable for import into statistical software such as R, SPSS, or Python's pandas library.

Q: Is TRI-CA suitable for fully remote/online experiments?

Yes. TRI-CA is a web-based platform and supports fully online experiments. Participants access the experiment via a URL with no software installation required on their end.

15. Troubleshooting

Installation Issues

npm install fails

Ensure Node.js v18 or later is installed. Run node --version and npm --version to confirm. If dependencies still fail, delete the node_modules/ folder and package-lock.json and try again.

Git clone fails

Verify your Git installation and network connection. Ensure you have the correct repository URL.

Server & Deployment Issues

Server does not start

Check that all required environment variables are set in your .env file. The most common causes are a missing OPENAI_API_KEY or invalid DATABASE_URL.

Participants cannot access the experiment URL

Verify the deployment completed successfully and the server is running. Check the Heroku (or host) logs for errors using heroku logs --tail.

Database Issues

Database connection error

Confirm that DATABASE_URL is correctly set. For Heroku, retrieve it with heroku config. Ensure the PostgreSQL add-on has been provisioned.

Data not being saved

Verify database write permissions and run a test participant session. Check server logs for any database write errors.

LLM / Agent Issues

Conversational agent not responding

Verify your API key is valid and has not exceeded usage limits. Check the LLM provider dashboard for any outages or quota warnings.

Agent behavior does not match the intended condition

Review the hidden_prompt field in your Treatment Group CSV and ensure the corresponding .txt file exists in prompt_bank/. Verify file names are spelled exactly as referenced.

Survey & Configuration Issues

Survey questions not displaying

Check the Question Bank CSV for formatting errors. Ensure each row has a valid block_name and question_name, and that the boolean fields (is_text, is_likert, etc.) contain valid values.

Treatment groups not assigning correctly

Verify the Treatment Group CSV is properly formatted and all referenced files (prompt files, task HTML files) exist in their respective directories.

16. Reproducibility & Extensibility

TRI-CA is designed to support transparent and reproducible research. Key design features include:

- Separation of experimental configuration from system code
- Version-controlled configuration files
- Automated logging of participant interactions
- Standardized data extraction scripts

Researchers can easily adapt the platform for new experiments by modifying configuration files without changing the underlying application architecture.